



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: **Básica**

Campo de Estudio: **Tronco común**

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): Carlos Jamil Papa Tanguila	
Asignatura: Estructura de Datos	Calificación:
Fecha del informe: 14/07/2026	
N.º Práctica: 2	
Título de la Práctica: Programación del botón Retroceder de un navegador web mediante una pila	
Introducción	Las estructuras de datos proporcionan reglas para organizar información y ejecutar operaciones sobre ella. Una pila trabaja con el principio LIFO (last in, first out): el último elemento incorporado es el primero que se retira. Este comportamiento se utiliza cuando una aplicación debe recuperar estados en orden inverso, como ocurre al deshacer acciones o regresar a la página visitada inmediatamente antes (Cormen et al., 2022; Jamro, 2024). En C#, <code>Stack<T></code> representa una colección LIFO de tamaño variable. <code>Push</code> inserta un elemento en la parte superior, <code>Pop</code> lo retira y devuelve, <code>Peek</code> permite consultarlo sin eliminarlo y <code>Count</code> informa cuántos elementos contiene la estructura. La documentación oficial señala que <code>Pop</code> tiene complejidad $O(1)$, mientras <code>Push</code> es $O(1)$ cuando existe capacidad disponible y puede llegar a $O(n)$ si el arreglo interno necesita ampliarse (Microsoft, s. f.-a). El problema seleccionado fue la programación del botón Retroceder de un navegador web. La necesidad consiste en conservar las páginas anteriores para recuperarlas desde la más reciente hasta la más antigua. Los usuarios son las personas que navegan entre páginas; los procesos principales son visitar, almacenar el estado anterior, visualizar el historial, consultar una página y retroceder. Por esta razón se eligió una pila y no una cola: una cola devolvería primero la página más antigua, lo cual no corresponde al

	comportamiento esperado del botón. La solución se desarrolló en C# con programación orientada a objetos. PaginaWeb representa cada registro mediante título, URL, categoría y fecha de visita; Navegador encapsula la pila y las operaciones del sistema; y Program gestiona el menú de consola. El objetivo fue implementar y comprobar un historial LIFO funcional, con reportería legible, consulta de los elementos almacenados, control de pila vacía y evidencias del flujo real de ejecución.
Desarrollo o procedimiento	Escenario y recursos La práctica se realizó de forma individual en un entorno doméstico de estudio, utilizando un computador con Windows, Visual Studio Code, el SDK de .NET, una terminal PowerShell, Git y GitHub. La documentación de Microsoft Learn se consultó para verificar el funcionamiento de Stack<T>. Las pruebas utilizaron Google, Microsoft Learn y Moodle UEA porque permiten observar con claridad el orden de navegación. Identificación del problema Necesidad: recuperar la página visitada inmediatamente antes. Usuario: persona que interactúa con el navegador simulado. Entrada: título, URL y categoría de una nueva página. Proceso: guardar la página actual con Push y asignar la nueva. Salida: página actual, historial disponible, consultas, reportes y mensajes de validación. Diseño orientado a objetos La clase PaginaWeb contiene los atributos tipados Titulo, Url y Categoria de tipo string, además de FechaVisita de tipo DateTime. Su constructor inicializa el objeto y MostrarDetalle presenta un registro completo. La clase Navegador mantiene PaginaActual, una pila privada Stack<PaginaWeb> denominada historialRetroceso y una lista auxiliar List<PaginaWeb> llamada registroVisitas. La pila es la estructura principal: controla exclusivamente el botón Retroceder. La lista solo conserva el reporte acumulado después de que los elementos salen de la pila. Navegador implementa Visitar, Retroceder, MostrarHistorialRetroceso, MostrarReporteVisitas y BuscarEnHistorialRetroceso. Esta distribución encapsula el estado y evita que Program modifique directamente las colecciones. Program únicamente captura entradas, valida datos y llama a los métodos correspondientes, coherente con la separación de responsabilidades de la programación orientada a objetos (Microsoft, 2025).

	Procedimiento aplicado 1. Se creó y verificó el proyecto de consola Practico_experimental2. 2. Se implementó PaginaWeb para representar cada página como un objeto con datos propios. 3. Se implementó Navegador y se inicializó la página de inicio. 4. Al visitar una página, Visitar ejecuta Push sobre la página actual, cambia PaginaActual y agrega el registro al reporte auxiliar. 5. Al retroceder, Retroceder verifica Count; si la pila contiene elementos, ejecuta Pop y devuelve true. Si está vacía, devuelve false sin producir una excepción. 6. La reportería enumera la pila sin modificarla y muestra el orden disponible para retroceder. La consulta recorre esa misma pila y compara título, URL o categoría. 7. Se construyó un menú de seis opciones y se validaron campos vacíos, opciones no numéricas, consultas sin coincidencias y retroceso con pila vacía. 8. Se ejecutó el flujo completo, se tomaron capturas y se publicó la solución en GitHub. Agente de inteligencia artificial utilizado Se utilizó Codex para interpretar la guía, revisar la rúbrica, explicar errores, proponer la estructura de clases, redactar y corregir código, orientar las pruebas y apoyar la elaboración del informe. El estudiante seleccionó el problema, creó y ejecutó el proyecto, ingresó los datos de prueba, verificó el comportamiento, tomó las capturas y revisó los resultados. Debido a la intervención directa de Codex en la versión final, se estima que aproximadamente el 85 % del código recibió asistencia de IA. El porcentaje corresponde a una estimación basada en el proceso de elaboración.
Resultados	La aplicación compiló sin errores ni advertencias. Se registraron Google, Microsoft Learn y Moodle UEA. Con Moodle UEA como página actual, el historial mostró primero Microsoft Learn, luego Google y finalmente Página de inicio. El resultado comprobó el comportamiento LIFO: el último estado almacenado fue el primero que se recuperó. La reportería presentó los tres objetos contenidos en el historial de retroceso con título y URL. La consulta de la pila encontró Microsoft Learn por su título y mostró también URL, categoría y fecha. El reporte acumulado presentó las tres páginas visitadas y el total de registros. Las capturas demuestran que los resultados son legibles y que la consulta no altera el orden de la pila. Al presionar Retroceder se recuperó Microsoft Learn, después

Google y finalmente Página de inicio. Un intento adicional devolvió el mensaje 'No hay páginas anteriores en el historial', sin generar InvalidOperationException. También se comprobó la salida controlada mediante la opción 6.

Análisis del tiempo de ejecución

Operación	Acción principal	Tiempo	Justificación
Visitar	Push + asignación	$O(1)$ amortizado	Push puede ser $O(n)$ si la pila amplía su arreglo interno.
Retroceder	Count + Pop	$O(1)$	Solo consulta y retira el elemento superior.
Mostrar historial	Recorrido foreach	$O(n)$	Debe visitar cada página almacenada.
Memoria	n páginas guardadas	$O(n)$	Cada visita anterior ocupa una posición en la pila.

La consulta BuscarEnHistorialRetroceso tiene complejidad $O(n)$, porque en el peor caso debe revisar todos los objetos de la pila. MostrarReporteVisitas también es $O(n)$. Para la secuencia de tres páginas, las operaciones fueron inmediatas; sin embargo, el análisis asintótico permite anticipar el crecimiento cuando aumenta el historial.

Análisis crítico de la pila en este problema

Ventajas: la pila reproduce exactamente el retroceso del navegador; Push y Pop son operaciones simples y eficientes; el elemento superior representa siempre la página anterior inmediata; y Count permite detectar el historial vacío antes de retirar un elemento. La enumeración ofrece reportería sin alterar el contenido.

Desventajas: solo se accede directamente al elemento superior; consultar una página intermedia requiere $O(n)$; al retirar una página con Pop deja de estar disponible en la pila; el historial consume memoria $O(n)$; y una sola pila no permite implementar el botón Adelante. Para esa ampliación sería necesaria una segunda pila. Además, el historial se pierde al cerrar el programa porque no existe persistencia en archivo o base de datos.

La elección queda justificada porque el requisito central no consiste en atender elementos por orden de llegada, sino recuperar primero el estado más reciente. Una cola FIFO sería correcta para filas de personas o asignación de asientos, pero devolvería la página más antigua y produciría un retroceso incorrecto en este escenario.

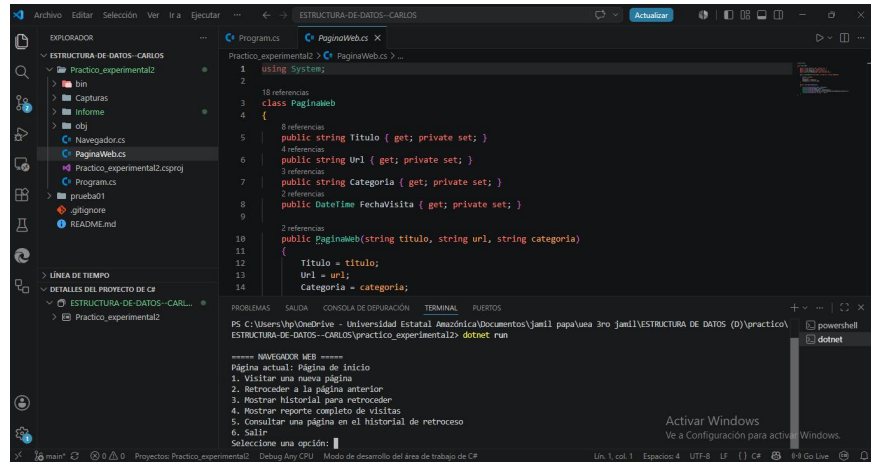
	Repositorio: https://github.com/CarlosPapa18/ESTRUCTURA-DE-DATOS--CARLOS/tree/main/Practico_experimental2
Conclusiones	Se cumplió el objetivo de implementar el botón Retroceder mediante una pila de objetos PaginaWeb. La secuencia Moodle UEA, Microsoft Learn, Google y Página de inicio confirmó que Stack<PaginaWeb> conserva y recupera estados en orden LIFO, que es el orden requerido por un navegador. La organización en PaginaWeb, Navegador y Program permitió separar los datos de cada página, las reglas del historial y la interacción con el usuario. La verificación de Count antes de Pop evitó errores al vaciar la pila, mientras la reportería y la consulta demostraron que sus elementos pueden revisarse de forma clara sin modificar el orden almacenado. El análisis mostró que visitar tiene costo $O(1)$ amortizado y retroceder $O(1)$, por lo que la pila resulta eficiente para la operación principal. Sus limitaciones son el acceso restringido al elemento superior, la búsqueda lineal, la falta del botón Adelante y la ausencia de persistencia. Como trabajo futuro se propone incorporar una segunda pila para avanzar y guardar el historial en un archivo.
Bibliografía	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/ Jamro, M. (2024). C# data structures and algorithms: Harness the power of C# to build a diverse range of efficient applications (2nd ed.). Packt Publishing. https://www.packtpub.com/en-us/product/c-sharp-data-structures-and-algorithms-9781803248271 Microsoft. (s. f.-a). Stack<T> Clase (System.Collections.Generic). Microsoft Learn. Recuperado el 14 de julio de 2026, de https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.stack-1?view=net-10.0 Microsoft. (2025, 22 de abril). Clases, estructuras y registros (C#). Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/object-oriented/

Anexos

Evidencias del flujo real de ejecución

Figura 1.

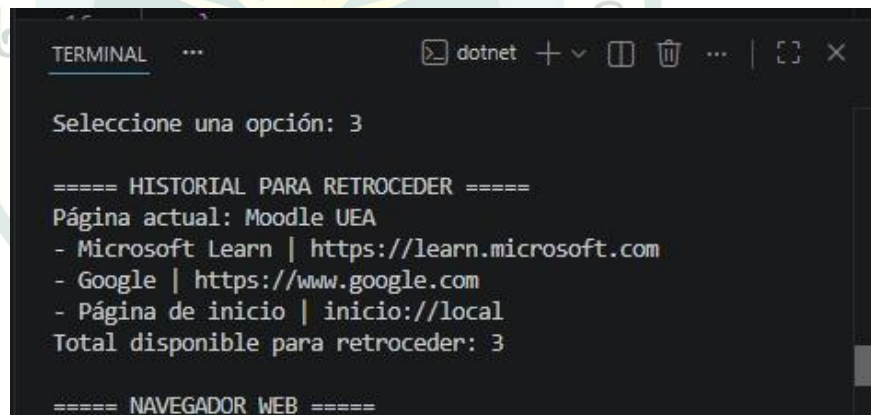
Definición de la clase PaginaWeb y menú principal del sistema



Nota. Captura de pantalla de elaboración propia realizada en Visual Studio Code (2026).

Figura 2.

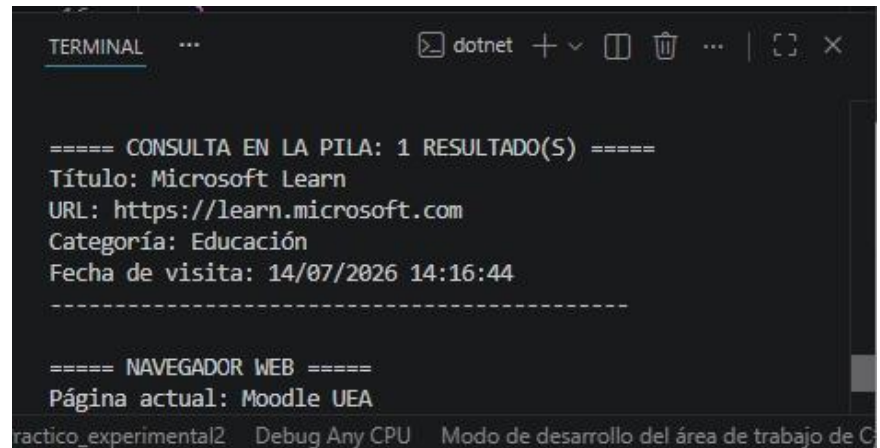
Visualización del historial de navegación mediante una pila LIFO



Nota. La captura muestra las páginas almacenadas en orden LIFO. Elaboración propia en Visual Studio Code (2026).

Figura 3.

Consulta de una página almacenada en la pila



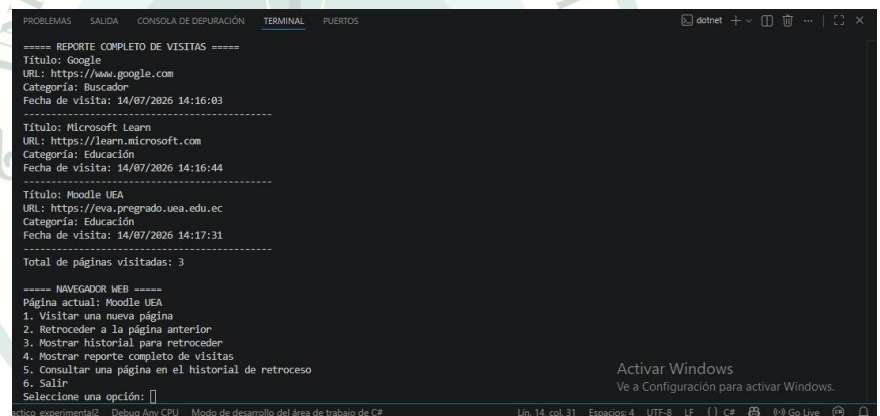
```
===== CONSULTA EN LA PILA: 1 RESULTADO(S) =====
Título: Microsoft Learn
URL: https://learn.microsoft.com
Categoría: Educación
Fecha de visita: 14/07/2026 14:16:44
-----

===== NAVEGADOR WEB =====
Página actual: Moodle UEA
```

Nota. Resultado de la búsqueda de Microsoft Learn dentro del historial de retroceso. Elaboración propia (2026).

Figura 4.

Reporte completo de las páginas visitadas



```
===== REPORTE COMPLETO DE VISITAS =====
Título: Google
URL: https://www.google.com
Categoría: Buscador
Fecha de visita: 14/07/2026 14:16:03
-----
Título: Microsoft Learn
URL: https://learn.microsoft.com
Categoría: Educación
Fecha de visita: 14/07/2026 14:16:44
-----
Título: Moodle UEA
URL: https://eva.pregado.uea.edu.ec
Categoría: Educación
Fecha de visita: 14/07/2026 14:17:31
-----
Total de páginas visitadas: 3

===== NAVEGADOR WEB =====
Página actual: Moodle UEA
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: []
```

Nota. El reporte presenta el título, la URL, la categoría y la fecha de visita de cada página registrada. Elaboración propia (2026).

Figura 5.

Funcionamiento de la operación de retroceso

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

===== NAVEGADOR WEB =====
Página actual: Moodle UEA
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: 2
Regresó a: Microsoft Learn
```

Nota. La captura evidencia el retorno desde Moodle UEA hacia Microsoft Learn mediante la operación Pop de la pila. Elaboración propia (2026).

Figura 6.

Validación del historial de navegación vacío

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

Seleccione una opción: 2
Regresó a: Microsoft Learn

===== NAVEGADOR WEB =====
Página actual: Microsoft Learn
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: 2
Regresó a: Google

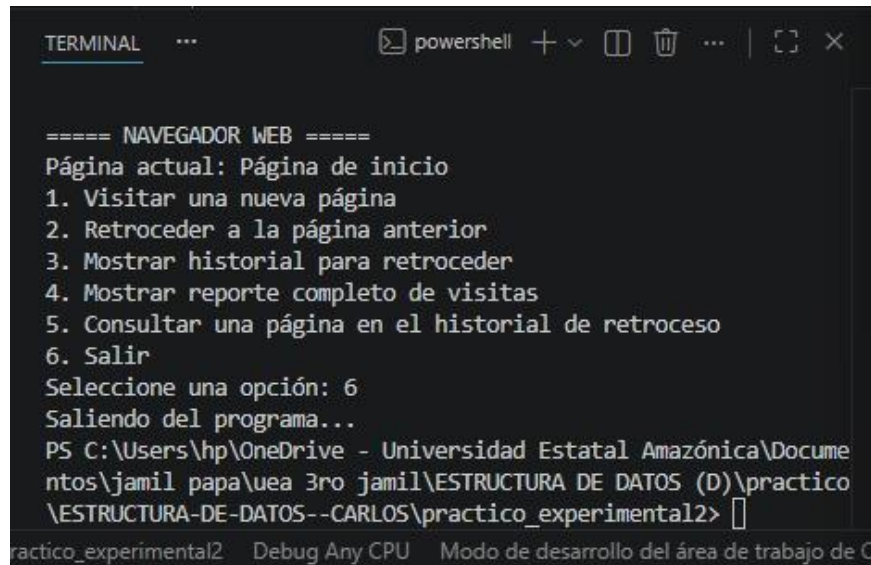
===== NAVEGADOR WEB =====
Página actual: Google
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: 2
Regresó a: Página de inicio

===== NAVEGADOR WEB =====
Página actual: Página de inicio
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: 2
No hay páginas anteriores en el historial.
```

Nota. El sistema informa que no existen páginas anteriores cuando la pila se encuentra vacía. Elaboración propia (2026).

Figura 7.

Finalización correcta de la aplicación



```
===== NAVEGADOR WEB =====
Página actual: Página de inicio
1. Visitar una nueva página
2. Retroceder a la página anterior
3. Mostrar historial para retroceder
4. Mostrar reporte completo de visitas
5. Consultar una página en el historial de retroceso
6. Salir
Seleccione una opción: 6
Saliendo del programa...
PS C:\Users\hp\OneDrive - Universidad Estatal Amazónica\Documen
tos\jamil papa\uea 3ro jamil\ESTRUCTURA DE DATOS (D)\practico
\ESTRUCTURA-DE-DATOS--CARLOS\practico_experimental2>
```

Nota. Captura de la opción de salida y del cierre controlado del programa.

Elaboración propia en Visual Studio Code (2026).

Código fuente completo

La versión ejecutada se incluye a continuación y coincide con el repositorio publicado.

Anexo 8. PaginaWeb.cs

```
using System;

class PaginaWeb
{
    public string Titulo { get; private set; }
    public string Url { get; private set; }
    public string Categoria { get; private set; }
    public DateTime FechaVisita { get; private set; }

    public PaginaWeb(string titulo, string url, string categoria)
    {
        Titulo = titulo;
        Url = url;
        Categoria = categoria;
        FechaVisita = DateTime.Now;
    }

    public void MostrarDetalle()
    {
        Console.WriteLine($"Título: {Titulo}");
        Console.WriteLine($"URL: {Url}");
        Console.WriteLine($"Categoría: {Categoria}");
        Console.WriteLine($"Fecha de visita: {FechaVisita:dd/MM/yyyy HH:mm:ss}");
        Console.WriteLine(new string('-', 45));
    }
}
```

Anexo 9. Navegador.cs

```
using System;
using System.Collections.Generic;

class Navegador
{
    private Stack<PaginaWeb> historialRetroceso;
    private List<PaginaWeb> registroVisitas;

    public PaginaWeb PaginaActual { get; private set; }

    public Navegador()
    {
```

```

        historialRetroceso = new Stack<PaginaWeb>();
        registroVisitas = new List<PaginaWeb>();
        PaginaActual = new PaginaWeb("Página de inicio", "inicio://local", "Sistema");
    }

    public void Visitar(PaginaWeb nuevaPagina)
    {
        historialRetroceso.Push(PaginaActual);
        PaginaActual = nuevaPagina;
        registroVisitas.Add(nuevaPagina);
    }

    public bool Retroceder()
    {
        if (historialRetroceso.Count == 0)
        {
            return false;
        }

        PaginaActual = historialRetroceso.Pop();
        return true;
    }

    public void MostrarHistorialRetroceso()
    {
        Console.WriteLine("\n===== HISTORIAL PARA RETROCEDER =====");
        Console.WriteLine($"Página actual: {PaginaActual.Titulo}");

        if (historialRetroceso.Count == 0)
        {
            Console.WriteLine("No hay páginas anteriores en el historial.");
            return;
        }

        foreach (PaginaWeb pagina in historialRetroceso)
        {
            Console.WriteLine($"- {pagina.Titulo} | {pagina.Url}");
        }

        Console.WriteLine($"Total disponible para retroceder: {historialRetroceso.Count}");
    }

    public void MostrarReporteVisitas()
    {
        Console.WriteLine("\n===== REPORTE COMPLETO DE VISITAS =====");

        if (registroVisitas.Count == 0)
        {
            Console.WriteLine("No se han registrado páginas visitadas.");
            return;
        }

        foreach (PaginaWeb pagina in registroVisitas)
        {
            pagina.MostrarDetalle();
        }

        Console.WriteLine($"Total de páginas visitadas: {registroVisitas.Count}");
    }

    public List<PaginaWeb> BuscarEnHistorialRetroceso(string texto)
    {
        List<PaginaWeb> resultados = new List<PaginaWeb>();

        foreach (PaginaWeb pagina in historialRetroceso)
        {
            bool coincideTitulo = pagina.Titulo.Contains(texto,
StringComparison.OrdinalIgnoreCase);
            bool coincideUrl = pagina.Url.Contains(texto,
StringComparison.OrdinalIgnoreCase);
            bool coincideCategoria = pagina.Categoria.Contains(texto,
StringComparison.OrdinalIgnoreCase);

            if (coincideTitulo || coincideUrl || coincideCategoria)
            {
                resultados.Add(pagina);
            }
        }

        return resultados;
    }
}

```

Anexo 10. Program.cs

```

using System;
using System.Collections.Generic;

Navegador navegador = new Navegador();
int opcion = 0;

```

```

do
{
    Console.WriteLine("\n===== NAVEGADOR WEB =====");
    Console.WriteLine($"Página actual: {navegador.PaginaActual.Titulo}");
    Console.WriteLine("1. Visitar una nueva página");
    Console.WriteLine("2. Retroceder a la página anterior");
    Console.WriteLine("3. Mostrar historial para retroceder");
    Console.WriteLine("4. Mostrar reporte completo de visitas");
    Console.WriteLine("5. Consultar una página en el historial de retroceso");
    Console.WriteLine("6. Salir");
    Console.Write("Seleccione una opción: ");

    if (!int.TryParse(Console.ReadLine(), out opcion))
    {
        Console.WriteLine("Debe ingresar un número.");
        continue;
    }

    switch (opcion)
    {
        case 1:
            Console.Write("Ingrese el título de la página: ");
            string titulo = Console.ReadLine() ?? "";
            Console.Write("Ingrese la URL: ");
            string url = Console.ReadLine() ?? "";
            Console.Write("Ingrese la categoría: ");
            string categoria = Console.ReadLine() ?? "";

            if (string.IsNullOrEmpty(titulo) ||
                string.IsNullOrEmpty(url) ||
                string.IsNullOrEmpty(categoria))
            {
                Console.WriteLine("Ningún dato puede quedar vacío.");
                break;
            }

            PaginaWeb nuevaPagina = new PaginaWeb(titulo, url, categoria);
            navegador.Visitar(nuevaPagina);
            Console.WriteLine($"Registro agregado. Visitando: {navegador.PaginaActual.Titulo}");
            break;

        case 2:
            if (navegador.Retroceder())
            {
                Console.WriteLine($"Regresó a: {navegador.PaginaActual.Titulo}");
            }
            else
            {
                Console.WriteLine("No hay páginas anteriores en el historial.");
            }
            break;

        case 3:
            navegador.MostrarHistorialRetroceso();
            break;

        case 4:
            navegador.MostrarReporteVisitas();
            break;

        case 5:
            Console.Write("Ingrese título, URL o categoría para consultar la pila: ");
            string textoBusqueda = Console.ReadLine() ?? "";

            if (string.IsNullOrEmpty(textoBusqueda))
            {
                Console.WriteLine("El texto de búsqueda no puede estar vacío.");
                break;
            }

            List<PaginaWeb> resultados = navegador.BuscarEnHistorialRetroceso(textoBusqueda);
            Console.WriteLine($"===== CONSULTA EN LA PILA: {resultados.Count} RESULTADO(S) =====");

            if (resultados.Count == 0)
            {
                Console.WriteLine("No se encontraron páginas que coincidan.");
            }
            else
            {
                foreach (PaginaWeb pagina in resultados)
                {
                    pagina.MostrarDetalle();
                }
            }
            break;

        case 6:
            Console.WriteLine("Saliendo del programa...");
            break;
    }
}

```

	<pre>default: Console.WriteLine("Opción inválida. Intente nuevamente."); break; } } while (opcion != 6);</pre>
--	--

Firma del estudiante/Firma de los estudiantes

